

سلام. امیدوارم حالتون خوب باشه. اینو برای گزارش فاز ۱ پروژه‌ی رمزنگاری می‌نویسم. قبل از هر چیزی نکته‌ای که وجود داره اینه که حدود ۶۰۰ خط کد شده و خب قطعا چون مقدارش خیلی زیاده نمی‌شه همشو به شکل خوبی توی گزارش نوشت. بخاطر همین هم من بخش‌های مهم‌ترشو اینجا توضیح می‌دم فقط و باقیشو می‌ذارم که توی تحویل توضیح بدم. بدون معطلی بیشتر بریم که شروع کنیم.

بخش اول) S-Box: تو این قسمت اومدم S-Box ها رو قرار دادم همشو به صورت هارد کد داخل کد گذاشتم. یه سری از S-Box ها داده نشده بودن که رفتم حسابشون کردم ولی محاسباتشو توی کد قرار ندادم. عکس یه دونه از این اس‌باکس‌ها رو این پایین می‌ذارم:



```

1 Sbox2 = [
2 ['224','5','88','217','103','78','129','203'],
3 ['201','11','174','106','213','24','93','130'],
4 ['70','223','214','39','138','50','75','66'],
5 ['219','28','158','156','58','202','37','123'],
6 ['13','113','95','31','248','215','62','157'],
7 ['124','96','185','190','188','139','22','52'],
8 ['77','195','114','149','171','142','186','122'],
9 ['179','2','180','173','162','172','216','154'],
10 ['23','53','204','247','153','97','90','232'],
11 ['36','86','64','48','99','9','70','86'],
12 ['191','152','151','133','104','252','236','10'],
13 ['218','111','83','98','163','46','8','175'],
14 ['40','176','116','194','189','54','34','56'],
15 ['100','30','57','44','166','48','229','68'],
16 ['253','136','159','101','135','107','244','35'],
17 ['72','16','209','81','192','249','210','160'],
18 ['85','161','65','250','67','19','196','47'],
19 ['168','182','60','43','193','255','200','160'],
20 ['32','137','0','144','71','239','234','183'],
21 ['21','6','205','181','18','126','187','41'],
22 ['15','184','7','4','155','148','33','102'],
23 ['230','206','237','231','59','254','127','197'],
24 ['164','55','177','76','145','110','141','118'],
25 ['3','45','222','150','38','125','198','92'],
26 ['211','242','79','25','63','220','121','29'],
27 ['8','27','235','243','109','94','121','211'],
28 ['240','49','12','212','207','140','226','117'],
29 ['169','74','87','132','17','69','27','245'],
30 ['228','14','115','170','241','221','89','20'],
31 ['108','146','8','208','120','112','48','2'],
32 ['128','80','167','246','119','147','134','131'],
33 ['42','199','91','233','238','143','1','61'],
34 ]
35

```

بخش دوم) توابع کمکی: من موقعی که داشتم کد می‌زدم تصمیم گرفتم که با اعداد باینری کار کنم. این یعنی اینکه تمام آپریشن‌ها روی اعداد باینری انجام می‌شه. تو ادامه‌ی مسیر این تصمیم فهمیدم که مثلا ایکس‌اور کردن روی دوتا عدد صحیح باید انجام بشه پس مثلا باید اول تبدیل بشن این اعداد باینری به عدد صحیح بعد ایکس‌اور بشن بعد دوباره تبدیل بشن به عدد باینری. درنتیجه مثلا برای همین کار اومدم یکی دوتا

تابع نوشتیم. حالا این یه مثال بود و کلا یه سری فانکشن کمکی نوشتیم که توی مسیر کمکم کنه. بخشی از این فانکشن‌ها رو اینجا عکسشونو می‌ذارم:

```
1 def string_to_padded_binary_array(input_str):
2     """
3     Convert a string to a binary array from its UTF-8 encoding
4     and pad the array
5     to ensure its size is a multiple of 16.
6     """
7     utf8_bytes = input_str.encode('utf-8')
8     binary_array = [format(byte, '08b') for byte in utf8_bytes]
9
10    padding_length = (16 - len(binary_array) % 16) % 16
11    binary_array.extend(['00000000'] * padding_length)
12    return binary_array
13
14 def binary_array_to_integer(binary_array):
15     """
16     Convert an array of binary strings into a single integer.
17     """
18    combined_binary = ''.join(binary_array)
19    result = int(combined_binary, 2)
20    return result
```

```
1 def binary_with_pad(num, length=32):
2     """
3     Convert an integer to a binary string, padded with leading zeros to the
4     specified length.
5     """
6    return bin(num)[2:].zfill(length)
7
8 def binary_xor_array(arr1, arr2):
9     return [bin(int(arr1[i], 2) ^ int(arr2[i], 2))[2:].zfill(len(arr1[i]))
10             for i in range(len(arr1))]
```

بخش سوم) جابه‌جایی داخل S-Box: به فانکشن نوشتن که با به منطق خیلی ساده میاد عمل جابه‌جایی رو انجام می‌ده. به عدد باینری می‌گیره بعد به ۵ بیت اولش نگاه می‌کنه که سطرش معلوم شه و بعد به ۳ بیت آخرش نگاه می‌کنه که ستونش معلوم شه و بعد هم عنصر داخل اون سطر و ستون رو خروجی می‌ده. کدشو این پایین می‌ذارم:

```
1 def change_of_Sbox(character,  
2   number_of_Sbox):  
3     row = int(character[0:5], 2)  
4     col = int(character[5:], 2)  
5     match number_of_Sbox:  
6         case 1:  
7             return Sbox1[row][col]  
8         case 2:  
9             return Sbox2[row][col]  
10        case 3:  
11            return Sbox3[row][col]  
12        case 4:  
13            return Sbox4[row][col]  
14        case 5:  
15            return Sbox5[row][col]  
16        case 6:  
17            return Sbox6[row][col]  
18        case 7:  
19            return Sbox7[row][col]  
20        case 8:  
21            return Sbox8[row][col]  
22
```

بخش چهارم) ماژول‌های مدیریت کلید: این قسمت خودش دو بخش داره: ۱) pre-key و ۲) key-gen. توی قسمت pre-key ما کلید ۱۲۸ بیتیمون رو می‌دیم بهش و به خروجی بهمون می‌ده اون خروجی می‌ره به key gen و بخش دوم pre key. بعد خروجی pre key دوم می‌ره به به key gen دیگه و یک دور از زیرکلیدهامون تولید می‌شه. توی هر دور، ۴ تا از زیر کلیدهامون که ۶۴ بیت هم هستن تولید می‌شن و کلاً ۴ دور هم این بخش تکرار می‌شه که در مجموع می‌شه ۱۶ تا زیرکلید ۶۴ بیتی. فرمول‌ها و شکلش داخل پی‌دی‌اف پروژه بود و من برای پیاده‌سازی دقیقاً همونجا رو دیدم فقط. به بخشی از کد این قسمت هم این پایین می‌ذارم:

```
1 def key_gen1(z):  
2     k_first = int(change_of_Sbox(z[8], 5), 16) ^ int(change_of_Sbox(z[9], 6), 16) ^ int(change_of_Sbox(z[7], 7), 16) ^ int(change_of_Sbox(z[6], 8), 16) ^ int(change_of_Sbox(z[2], 5), 16)  
3     k_second = int(change_of_Sbox(z[10], 5), 16) ^ int(change_of_Sbox(z[11], 6), 16) ^ int(change_of_Sbox(z[5], 7), 16) ^ int(change_of_Sbox(z[4], 8), 16) ^ int(change_of_Sbox(z[6], 6), 16)  
4     K1 = (bin(k_first)[2:] + bin(k_second)[2:])  
5     k_third = int(change_of_Sbox(z[12], 5), 16) ^ int(change_of_Sbox(z[13], 6), 16) ^ int(change_of_Sbox(z[3], 7), 16) ^ int(change_of_Sbox(z[2], 8), 16) ^ int(change_of_Sbox(z[9], 7), 16)  
6     k_forth = int(change_of_Sbox(z[14], 5), 16) ^ int(change_of_Sbox(z[15], 6), 16) ^ int(change_of_Sbox(z[1], 7), 16) ^ int(change_of_Sbox(z[0], 8), 16) ^ int(change_of_Sbox(z[13], 8), 16)  
7     K2 = (bin(k_third)[2:] + bin(k_forth)[2:])  
8     return K1, K2  
9
```

```

1 def pre_key1(x):
2     z = [''] * 16
3     z1 = binary_array_to_integer(x[0:4]) ^ int(change_of_Sbox(x[13], 5), 16) ^
int(change_of_Sbox(x[15], 6), 16) ^ int(change_of_Sbox(x[12], 7), 16) ^ int(
change_of_Sbox(x[14], 8), 16) ^ int(change_of_Sbox(x[8], 7), 16)
4     z[0], z[1], z[2], z[3] = chunking_binary(bin(z1)[2:].zfill(32))
5     z2 = binary_array_to_integer(x[8:12]) ^ int(change_of_Sbox(z[0], 5), 16) ^
int(change_of_Sbox(z[2], 6), 16) ^ int(change_of_Sbox(z[1], 7), 16) ^ int(
change_of_Sbox(z[3], 8), 16) ^ int(change_of_Sbox(x[10], 8), 16)
6     z[4], z[5], z[6], z[7] = chunking_binary(bin(z2)[2:].zfill(32))
7     z3 = binary_array_to_integer(x[12:16]) ^ int(change_of_Sbox(z[7], 5), 16) ^
int(change_of_Sbox(z[6], 6), 16) ^ int(change_of_Sbox(z[5], 7), 16) ^ int(
change_of_Sbox(z[4], 8), 16) ^ int(change_of_Sbox(x[9], 5), 16)
8     z[8], z[9], z[10], z[11] = chunking_binary(bin(z3)[2:].zfill(32))
9     z4 = binary_array_to_integer(x[4:8]) ^ int(change_of_Sbox(z[10], 5), 16) ^
int(change_of_Sbox(z[9], 6), 16) ^ int(change_of_Sbox(z[11], 7), 16) ^ int(
change_of_Sbox(z[8], 8), 16) ^ int(change_of_Sbox(x[11], 6), 16)
10    z[12], z[13], z[14], z[15] = chunking_binary(bin(z4)[2:].zfill(32))
11    return z
12

```

```

1 def generate_keys(x):
2     z = pre_key1(x)
3     x = pre_key2(z)
4     K1,K2 = key_gen1(z)
5     K3,K4 = key_gen2(x)
6     z = pre_key1(x)
7     x = pre_key2(z)
8     K5,K6 = key_gen3(z)
9     K7,K8 = key_gen4(x)
10    z = pre_key1(x)
11    x = pre_key2(z)
12    K9,K10 = key_gen1(z)
13    K11,K12 = key_gen2(x)
14    z = pre_key1(x)
15    x = pre_key2(z)
16    K13,K14 = key_gen3(z)
17    K15,K16 = key_gen4(x)
18    return [K1,K2,K3,K4,K5,K6,K7,K8,K9,K10,K11,K12,
K13,K14,K15,K16]
19

```

بخش پنجم) تابع دور: اینجا قلب اصلی رمزنگاری ما هست. اینجا ما میایم و مطابق شکلی که داخل پی‌دی‌اف هست خود تابع رو با زیرکلیدهایی که از مرحله‌ی قبلی تولید کردیم پیاده‌سازی می‌کنیم. این تابع دوتا ورودی داره. یکی متنی که می‌خواهیم رمز کنیم و دومی هم زیرکلید. نکته‌ی آخر هم اینکه چون ضرب ماتریسی با numpy array راحت‌تره من از اون استفاده کردم. کد این بخش هم این پایین می‌ذارم:

```
1 def round_function(sub_key, x):
2     z1 = int(x[0], 2) ^ int(sub_key[7], 2)
3     z2 = int(x[1], 2) ^ int(sub_key[6], 2)
4     z3 = int(x[2], 2) ^ int(sub_key[5], 2)
5     z4 = int(x[3], 2) ^ int(sub_key[4], 2)
6     z5 = int(x[4], 2) ^ int(sub_key[3], 2)
7     z6 = int(x[5], 2) ^ int(sub_key[2], 2)
8     z7 = int(x[6], 2) ^ int(sub_key[1], 2)
9     z8 = int(x[7], 2) ^ int(sub_key[0], 2)
10
11     z1 = int(change_of_Sbox(bin(z1)[2:].zfill(8), 1), 10)
12     z2 = int(change_of_Sbox(bin(z2)[2:].zfill(8), 2), 10)
13     z3 = int(change_of_Sbox(bin(z3)[2:].zfill(8), 3), 10)
14     z4 = int(change_of_Sbox(bin(z4)[2:].zfill(8), 4), 10)
15     z5 = int(change_of_Sbox(bin(z5)[2:].zfill(8), 2), 10)
16     z6 = int(change_of_Sbox(bin(z6)[2:].zfill(8), 3), 10)
17     z7 = int(change_of_Sbox(bin(z7)[2:].zfill(8), 4), 10)
18     z8 = int(change_of_Sbox(bin(z8)[2:].zfill(8), 1), 10)
19     coef = np.array([
20         [0,1,1,1,1,0,0,1],
21         [1,0,1,1,1,1,0,0],
22         [1,1,0,1,0,1,1,0],
23         [1,1,1,0,0,0,1,1],
24         [0,1,1,1,1,1,1,0],
25         [1,0,1,1,0,1,1,1],
26         [1,1,0,1,1,0,1,1],
27         [1,1,1,0,1,1,0,1],])
28
29     Z = np.array([z1,z2,z3,z4,z5,z6,z7,z8])
30     Z = [bin(i)[2:].zfill(8) for i in Z]
31     Z = [[int(i) for i in j] for j in Z]
32     result = np.dot(coef,Z)
33     result = [i % 2 for i in result]
34     result = [''.join([str(i) for i in row])[2:] for row
35         in result]
36     return result
```

بخش ششم) تابع رمزگذاری و رمزگشایی: توی این بخش اومدم و دوتا تابع جدا برای هر کدوم از اینا نوشتم که البته خیلی هم شبیه هم هستن ولی یکی رمزگشایی و می‌کنه و اون یکی رمزگشایی. هر دوتاشون ۲تا ورودی می‌گیرن. ورودی اولشون متنی که قراره رمزگذاری یا رمزگشایی بشه هست و ورودی دومشون کل زیرکلیدهای تولید شده هست. داخل هر کدومشون یه حلقه‌ی ۱۶تایی هست که ۱۶ مرحله کدگذاری یا کدگشایی با استفاده از زیرکلید مربوط به اون دور رو انجام می‌ده. کد این بخش هم این پایین می‌ذارم:

```

1 def encryption_feistel(text, key):
2     left = text[:8]
3     right = text[8:]
4     for i in range(16):
5         round_key = key[i]
6         right_binary = ''.join(right)
7         # Join the array into a single binary string
8         if len(right_binary) < 64:
9             right_binary = right_binary.zfill(64)
10        # Pad with leading zeros if necessary
11
12        # Split the 64-bit binary string back into 8-bit chunks
13        right = [right_binary[i:i+8] for i in range(0, 64, 8)]
14        after_round_function = round_function(round_key, right)
15        after_xor = binary_array_to_integer(left) ^
16        binary_array_to_integer(after_round_function)
17        new_right = int_to_binary_array(after_xor)
18        left, right = right, new_right
19    return right + left

```

```

1 def decryption_feistel(cypher_text, key):
2     left = cypher_text[:8]
3     right = cypher_text[8:]
4     for i in range(16):
5         round_key = key[(15-i)]
6         right_binary = ''.join(right)
7         # Join the array into a single binary string
8         if len(right_binary) < 64:
9             right_binary = right_binary.zfill(64)
10        # Pad with leading zeros if necessary
11
12        # Split the 64-bit binary string back into 8-bit chunks
13        right = [right_binary[i:i+8] for i in range(0, 64, 8)]
14        after_round_function = round_function(round_key, right)
15        after_xor = binary_array_to_integer(left) ^
16        binary_array_to_integer(after_round_function)
17        new_right = int_to_binary_array(after_xor)
18        left, right = right, new_right
19    return right + left

```

بخش هفتم) مودهای مختلف رمزگذاری و رمزگشایی: توی این بخش اومدم مودهای مختلفی که داخل صورت پروژه گفته شده بود (EBC، CBC، CTR، OFB) رو پیاده‌سازی کردم. مثلاً توی مود EBC ما میایم هر تیکه رو فارغ از بقیه رمزگذاری و رمزگشایی می‌کنیم و برای رمزگذاری یا رمزگشایی یک بلاک هیچ نیازی به یک مقدار قبلی یا مقدار اولیه وجود نداره. دقیقاً همین موضوع داخل کدش هم هست و داخل کد هم صرفاً

کدگذاری می‌کنیم و بعد داخل متن رمز قرار می‌دهیم. کدشو این پایین قرار دادم. توضیح بقیه‌ی بخش‌ها هم داخل تحویل اگر خواسته شد، می‌گم. کد EBC رو این پایین قرار دادم:

```
1 def EBC_mode_encryption(plain_text, key):
2     plain_text = string_to_padded_binary_array(plain_text)
3     plain_text = [plain_text[i:i+16] for i in range(0, len(
4         plain_text), 16)]
5     cipher_chunks = []
6
7     for plain_chunk in plain_text:
8         keys = generate_keys(key)
9         cipher = encryption_feistel(plain_chunk, keys)
10        cipher_chunks.append(cipher)
11
12    final_cipher = flat_arr(cipher_chunks)
13    return final_cipher
```

```
1 def EBC_mode_decryption(cipher_text, key):
2     cipher_text = [cipher_text[i:i+16] for i in range(0,
3         len(cipher_text), 16)]
4     plain_chunks = []
5
6     for cipher_chunk in cipher_text:
7         keys = generate_keys(key)
8         plain = decryption_feistel(cipher_chunk, keys)
9         plain_chunks.append(plain)
10
11    final_plain = flat_arr(plain_chunks)
12    return final_plain
```

بخش هشتم) تست و خروجی: تو این بخش هم اوادم و با مود رمزنگاری EBC به متن رو رمزگذاری کردم و بعد رمزگشاییش کردم که کد و خروجی هم این پایین قرار دادم:

```

1 # Example usage
2 x = ['10010110', '1011010', '11110000', '01110100', '00101001', '01100010', '00110111', '11100010', '00010110', '10000010', '10000001', '01100011', '00000110', '00001010', '00010100', '11011000']
3 initial = ['10101000'] * 16
4 cipher_text = EBC_mode_encryption("
    hello this is a test and i want this text to be more long enough so i can test my code", x)
5 print(f"this is binary of encrypted: {cipher_text} and len of it is: {len(cipher_text)}")
6 print("Cipher Text:", decode_binary_array_with_errors(cipher_text))
7
8 plain_text = EBC_mode_decryption(cipher_text, x)
9 print(f"this is binary of decrypted: {plain_text} and len of it is: {len(plain_text)}")
10 print("Decrypted Text:", decode_binary_array_with_errors(plain_text))
11

```

```

Cipher Text: is␣p␣he␣b␣U␣ i␣*␣G␣te␣*␣t␣ y␣E␣*␣th1␣y␣(␣g␣ ,␣J␣t␣mok␣!
eTn ␣*␣*␣*␣*␣ sX␣*␣4D␣
␣QA,my xx␣g␣
his is binary of decrypted: ['1011010001', '1011001011', '1011011001', '10

```

```

Decrypted Text: hello this is a test and i want this text to be more long enough so i can test my code

```

خب این گزارش من از بود از پروژه فاز اول. امیدوارم که خوشتون اومده باشه.